

AsyncVLA: Asynchronous Flow Matching for Vision-Language-Action Models

Yuhua Jiang^{1,2}, Shuang Cheng^{2,3}, Yan Ding⁴, Feifei Gao^{1†}, Biqing Qi^{2†}

¹Tsinghua University, ²Shanghai AI Laboratory, ³Zhejiang University, ⁴Lumos Robotics

Abstract

Vision-language-action (VLA) models have recently emerged as a powerful paradigm for building generalist robots. However, traditional VLA models that generate actions through flow matching (FM) typically rely on rigid and uniform time schedules, i.e., synchronous FM (SFM). Without action context awareness and asynchronous self-correction, SFM becomes unstable in long-horizon tasks, where a single action error can cascade into failure. In this work, we propose asynchronous flow matching VLA (AsyncVLA), a novel framework that introduces temporal flexibility in asynchronous FM (AFM) and enables self-correction in action generation. AsyncVLA breaks from the vanilla SFM in VLA models by generating the action tokens in a non-uniform time schedule with action context awareness. Besides, our method introduces the confidence rater to extract confidence of the initially generated actions, enabling the model to selectively refine inaccurate action tokens before execution. Moreover, we propose a unified training procedure for SFM and AFM that endows a single model with both modes, improving KV-cache utilization. Extensive experiments on robotic manipulation benchmarks demonstrate that AsyncVLA is data-efficient and exhibits self-correction ability. AsyncVLA outperforms existing methods across both simulation and real-world evaluations. Code is available at <https://github.com/YuhuaJiang2002/AsyncVLA>.

1 Introduction

Training generalist robot policies that integrate perception, language, and low-level control remains a central challenge for embodied intelligence [Reed et al., 2022, Liu et al., 2024, Ma et al., 2024, Sapkota et al., 2025, Huang et al., 2024, Li et al., 2025c, 2024a, Chi et al., 2023]. Vision-language-action (VLA) models address this by leveraging large-scale vision-language corpora and robot demonstrations to ground broad semantics into executable control [Brohan et al., 2022, Huang et al., 2023, Bousmalis et al., 2023, Ahn et al., 2022, Fu et al., 2024, Yu et al., 2025, Zang et al., 2025, Liu et al., 2025]. To improve success rates and efficiency, follow-up work advances architectures and action generation through better spatiotemporal modeling [Li et al., 2025d, Chen et al., 2025b, Lv et al., 2025, Wu et al., 2025, Shi et al., 2025], parameter-efficient adaptation [Wang et al., 2025], improved tokenization [Pertsch et al., 2025, Goyal et al., 2025], high-throughput execution [Kim et al., 2025, Shukor et al., 2025], and interleaving intermediate reasoning with action prediction [Wen et al., 2025b, Zhao et al., 2024, Zhong et al., 2025, Huang et al., 2025b, Wen et al., 2025a]. Building on these advances, recent VLA models incorporate self-correction to improve reliability under uncertainty. Imitating fast and slow systems in the human brain [Li et al., 2025e, Jiang et al., 2025], SC-VLA [Li et al., 2025b] pairs a fast action head with a slow self-correction module to detect failures. Enhanced by reinforcement learning, RB-VLA [Li et al., 2025a] applies a dual-pathway loop for in-situ adaptation. Inspired by diffusion large language models (DLLMs) [Nie et al., 2025, Swerdlow et al., 2025, Cheng et al., 2025], discrete-diffusion VLA [Liang et al., 2025] brings masked-token denoising and secondary remasking into VLA for adaptive decoding. LLaDA-VLA [Wen et al.,

2025c], dVLA [Wen et al., 2025a], and UD-VLA [Chen et al., 2025a] further employ multi-modal chain of thought (CoT) with prefix attention and KV-cache.

Despite these successes, a fundamental limitation exists in mainstream VLA architectures: their reliance on a rigid and synchronous action generation process [Black et al., 2024, 2025, Zhai et al., 2025, Qu et al., 2025a]. VLA models based on vanilla flow matching (FM) employ a uniform time schedule across all action tokens, generating them synchronously from noise to the final actions, i.e., synchronous FM (SFM). SFM employs fixed action-generation time schedules, regardless of the task’s current complexity or the model’s internal confidence [Community, 2025, Zhang et al., 2024, Dong et al., 2025]. Without the utilization of action context information and the mechanism for self-correction, SFM’s monolithic generation method is inherently unstable. Consequently, a single inaccurate action prediction can cascade into an unrecoverable error, critically hindering performance in long-horizon or precision-demanding scenarios.

In order to utilize the action context information in action generation, we find that temporal asynchrony—the ability to non-uniformly and dynamically decide the action generation time schedule—is the key to unlocking robust robotic control with self-correction ability. Our core insight is to re-frame action generation, particularly within the framework of FM [Lipman et al., 2022, Gao et al., 2025, Zheng et al., 2025a], not as a fixed procedure, but as a deliberative denoising process with asynchronous time schedule, where the model can reconsider those parts of the first-round generated actions with low confidence. By regenerating a subset of actions while keeping others unchanged, temporal asynchrony exploits the context information of first-round generated actions to refine potentially inaccurate actions, and thus realizes self-correction.

In this paper, we propose asynchronous flow matching VLA (AsyncVLA), a novel VLA framework that employs the initial SFM and the subsequent asynchronous FM (AFM), enabling confidence-aware robot action generation with self-correction. Instead of a fixed and uniform time schedule in the denoising process, AsyncVLA adaptively schedules its AFM time steps, performing regeneration on action tokens with low confidence. Specifically, we propose a confidence rater that evaluates the confidence of each action token generated by SFM. AsyncVLA leverages these confidence signals to trigger asynchronous self-correction, enabling the model to selectively revisit and refine low-confidence parts of its action plan before execution. Moreover, the first-round generated actions with relatively high confidence provide context information that facilitates correcting actions with relatively low confidence. Therefore, AsyncVLA possesses an introspective capability to dynamically modulate its generation process and selectively reconsider its generated actions based on confidence.

2 Related Work

Vision-Language-Action Models VLA models adapt VLM backbones to map visual observations and language instructions to low-level actions. Early works mainly use auto-regressive decoding with discretized action tokens [Kim et al., 2024, Brohan et al., 2022, Team et al., 2024, Goyal et al., 2025]. Inspired by CoT, CoT-VLA [Zhao et al., 2024] and FlowVLA [Zhong et al., 2025] generate future sub-goal images as visual CoT before predicting short action chunks, improving long-horizon success and interpretability. For efficiency, OpenVLA-OFT [Kim et al., 2025] enables parallel decoding with chunked control for high-throughput execution. For continuous actions, π_0 [Black et al., 2024], $\pi_{0.5}$ [Black et al., 2025], WALL-OSS [Zhai et al., 2025], and EO-1 [Qu et al., 2025a] adopt FM but rely on synchronous SFM schedules; in contrast, AsyncVLA combines AFM with confidence-driven self-correction to trigger calibrated regeneration only when needed.

Self-Correction in VLA Models Recent VLA models introduce self-correction to improve task success rates. CollabVLA [Sun et al., 2025] couples diffusion-based action generation with self-correction reasoning and requests human guidance under uncertainty, while ReflectVLM [Feng et al., 2025] removes human involvement via test-time reflection and diffusion-based imagination for iterative long-horizon plan revision. Motivated by fast-slow cognition [Li et al., 2025e], SC-VLA [Li et al., 2025b] integrates a fast action head with a slow correction module within one policy, and RB-VLA [Li et al., 2025a] further enables in-situ adaptation through a dual-pathway loop combining failure-driven and success-driven mechanism. Building on DLLMs [Nie et al., 2025, Swerdlow et al., 2025, Cheng et al., 2025], dVLA [Liang et al., 2025, Wen et al., 2025a], LLaDA-VLA [Wen et al., 2025c], and UD-VLA [Chen et al., 2025a] perform masked-token denoising for action-chunk generation with adaptive decoding and secondary remasking. However, these approaches mainly

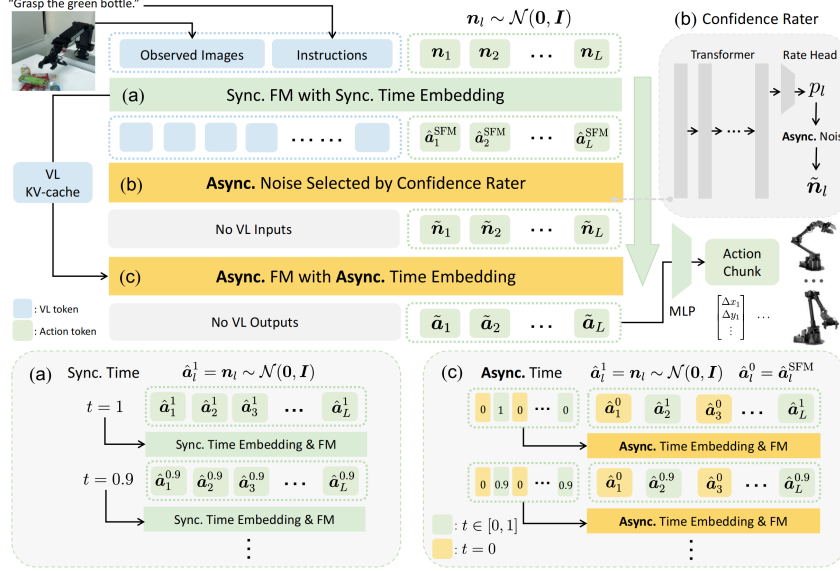


Figure 1: Overview of the AsyncVLA framework that comprises three components: (a) SFM applies a uniform time schedule t across all action tokens, generating them synchronously from noise ($t = 1$) to action ($t = 0$). (b) Confidence rater estimates the actions' token-level confidence and mask the low-confidence actions by selecting asynchronous noise for AFM. (c) AFM dynamically assigns individual FM time to each action token, allowing for selective and non-uniform regeneration based on the actions' confidence. SFM and AFM share a single unified model with the same parameters.

target self-correction for discrete action tokens. AsyncVLA proposes a unified SFM+AFM framework with confidence-guided regeneration, and extends self-correction to continuous action generation without large reward models.

3 Methodology

We introduce AsyncVLA, a VLA model enhanced by AFM. We start by introducing the self-correction mechanism of AFM in Sec. 3.1, followed by the confidence rater that determines the positions of masked action tokens in Sec. 3.2, and the overall training procedures in Sec. 3.3.

3.1 Asynchronous Flow Matching

We formulate the robot policy as a VLA model in a synergistic structure of VLM backbone and FM action head. The model can flexibly generate continuous action chunks whose length is denoted by L . The FM velocity for action generation can be written as $V_\theta(o_t, \ell, \hat{a}_{t:t+L}^\tau)$, where $o_t = [I_t^{(1)}, \dots, I_t^{(n)}, q_t]$ consists of multi-view image observations and robot state at time t , the language context ℓ is the embodied task instructions, and $\hat{a}_{t:t+L}^\tau$ is the partially denoised action chunk at FM time τ .

As shown in Figure 1, AsyncVLA consists of 3 sequential parts: SFM, the confidence rater, and AFM. SFM and AFM share the same model that is trained in a unified training procedure. For input, each token may correspond to a text token, an image patch token, a robot state token, or a partially denoised action token. For output, we employ an FM head to generate continuous action tokens.

Asynchronous Flow Matching Inference During inference of AFM, the model masks part of the action tokens generated by SFM, whose positions are denoted by the mask $m \in \{0, 1\}^L$. The element of m is 1 if the corresponding action token is masked and is 0 otherwise. In AFM generation, the unmasked tokens remain unchanged, while the masked tokens are updated using the forward Euler rule as:

$$\hat{a}_{t:t+L}^{\tau-\delta} = \hat{a}_{t:t+L}^\tau - \delta V_\theta(o_t, \ell, \hat{a}_{t:t+L}^\tau) \odot m, \quad (1)$$

where $\odot$ denotes the token-wise Hadamard product and δ denotes the time step size. For $\tau = 1$, we design the starting asynchronous noise $\hat{\mathbf{a}}_{t:t+L}^1 = [\tilde{\mathbf{n}}_{t+1}, \dots, \tilde{\mathbf{n}}_{t+L}]$ as:

$$\tilde{\mathbf{n}}_l = \begin{cases} \hat{\mathbf{a}}_l^{\text{SFM}}, & \text{if } m_l = 0, \\ \mathbf{n}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), & \text{if } m_l = 1, \end{cases} \quad (2)$$

where m_l is the l -th element of $\mathbf{m}$ with $l = t + 1, \dots, t + L$, and $\hat{\mathbf{a}}_l^{\text{SFM}}$ is the action predicted by the previous SFM. In AFM, the model incorporates information from SFM-estimated action tokens even when the regenerated tokens remain in an early noisy stage, thus producing more accurate actions. Since SFM can be regarded as a fully-masked special case of AFM, we employ the same model for both SFM and AFM modes. Therefore, the VL KV-cache generated in SFM can be directly reutilized in AFM. In this way, we save the burden of repeatedly processing the VL tokens and thus significantly improve the model’s inference efficiency for real-time control. The procedure of AFM inference is summarized in Algorithm 1.

Asynchronous Time Embedding in AFM To distinguish masked from unmasked action tokens, we propose the asynchronous time embedding module. Denote the dimension of the VLM’s hidden states as d . At FM time τ , we apply the sinusoidal encoding function $\mathcal{S}(\cdot)$ to map $\tau\mathbf{m}$ to the asynchronous time-embedding matrix $\mathcal{S}(\tau\mathbf{m}) \in \mathbb{R}^{L \times d}$. We then concatenate $\mathcal{S}(\tau\mathbf{m})$ and the linearly projected noisy action $\mathcal{P}(\hat{\mathbf{a}}_{t:t+L}^\tau) \in \mathbb{R}^{L \times d}$ along the last dimension and yield $\hat{\mathbf{h}}_{t:t+L}^\tau \in \mathbb{R}^{L \times 2d}$. Finally, a multi-layer perceptron (MLP) is utilized to project $\hat{\mathbf{h}}_{t:t+L}^\tau$ to the asynchronous time-embedded action hidden state $\hat{\mathbf{x}}_{t:t+L}^\tau \in \mathbb{R}^{L \times d}$. With the same hidden dimension as the VLM, $\hat{\mathbf{x}}_{t:t+L}^\tau \in \mathbb{R}^{L \times d}$ is concatenated with VL hidden states and then sent into the VLM’s transformer backbone. Following [Zhao et al., 2024], we apply full attention for action generation.

3.2 Confidence Rater

Since AsyncVLA lacks a dedicated output head for action-token logits, it is hard to directly estimate the model’s confidence based on token probability. Thus, we individually design a confidence rater to estimate the confidence of the actions. The confidence rater takes the final-layer hidden states of VL tokens concatenated with the first-round actions generated by SFM as input, and evaluates the confidence of the l -th action token as $p_l \in (0, 1)$, $l = t + 1, \dots, t + L$.

The confidence rater consists of several transformer layers and a final linear layer as its rate head. The action tokens are projected into the embedding space of VL tokens using a linear layer. The transformer layers apply full attention, such that the confidence can be calculated according to the VL information and the context actions before or after the evaluated action token. The rate head projects the hidden states to a scalar and employs Sigmoid function to generate p_l . Using p_l , we generate the l -th element of the mask as:

$$m_l = \mathbb{1}\{p_l < T\}, \quad l = t + 1, \dots, t + L, \quad (3)$$

where $\mathbb{1}\{\cdot\}$ is the indicator function and $T \in (0, 1)$ is a predefined confidence threshold that controls the number of masked tokens. In Equation (3), a self-adaptive number of action tokens will be masked according to the actions’ confidence, which offers better flexibility than other strategies such as Top-K selection. The confidence rater ensures that only actions with relatively large deviation are regenerated and that the unmasked actions providing context information are relatively accurate.

3.3 Training Procedures

Unified Training for SFM and AFM In order to employ a single model to realize both SFM and AFM inference, we propose a unified training procedure that treats SFM as a fully-masked special case of AFM. The VLM backbone and FM head are jointly trained by minimizing the following end-to-end AFM velocity prediction loss on masked tokens:

$$\mathcal{L} = \mathbb{E}_\tau \left\{ \left\| [V_\theta(\mathbf{o}_t, \ell, \hat{\mathbf{a}}_{t:t+L}^\tau) - \mathbf{u}_{t:t+L}] \odot \mathbf{m} \right\|^2 \right\}, \quad (4)$$

where $\mathbf{u}_{t:t+L} = \mathbf{n}_{t:t+L} - \mathbf{a}_{t:t+L}$ denotes the ground-truth velocity with Gaussian noise $\mathbf{n}_{t:t+L} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and $\hat{\mathbf{a}}_{t:t+L}^\tau$ denotes the intermediate asynchronous noisy action, computed as

$$\tilde{\mathbf{a}}_{t:t+L} = \mathbf{a}_{t:t+L} + \boldsymbol{\epsilon}_{t:t+L} \odot (\mathbf{1} - \mathbf{m}), \quad \boldsymbol{\epsilon}_{t:t+L} \sim \mathcal{N}(\mathbf{0}, \sigma_c^2 \mathbf{I}), \quad (5)$$

$$\hat{\mathbf{a}}_{t:t+L}^\tau = \tilde{\mathbf{a}}_{t:t+L} - \tau(\tilde{\mathbf{a}}_{t:t+L} - \mathbf{n}_{t:t+L}) \odot \mathbf{m}. \quad (6)$$

Table 1: LIBERO task performance results evaluated by success rates. OpenVLA-OFT (Con./Dis.) refers to OpenVLA-OFT with continuous or discrete action. All models are fine-tuned on the combined 4 task suites as a whole, instead of training 4 individual VLA models.

Model	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-Long	Avg.
WorldVLA [Cen et al., 2025]	87.6	96.2	83.4	60.0	81.8
TraceVLA [Zheng et al., 2025b]	84.6	85.2	75.1	54.1	74.8
SpatialVLA [Qu et al., 2025b]	88.2	89.9	78.6	55.5	78.1
π_0 -FAST [Pertsch et al., 2025]	96.4	96.8	88.6	60.2	85.5
π_0 [Black et al., 2024]	96.8	98.8	95.8	85.2	94.2
$\pi_{0.5}$ [Black et al., 2025]	98.8	98.2	98.0	92.4	96.9
OpenVLA-OFT (Con.) [Kim et al., 2025]	96.9	98.1	95.5	91.1	95.4
OpenVLA-OFT (Dis.) [Kim et al., 2025]	96.2	98.2	95.6	92.0	95.5
GR00T-N1 [NVIDIA et al., 2024]	94.4	97.6	93.0	90.6	93.9
UD-VLA [Chen et al., 2025a]	94.1	95.7	91.2	89.6	92.7
Discrete-Diffusion VLA [Liang et al., 2025]	97.2	98.6	97.4	92.0	96.3
dVLA [Wen et al., 2025a]	97.4	97.9	98.2	92.2	96.4
AsyncVLA (Ours)	98.4	99.2	98.6	93.4	97.4

where $\epsilon_{t:t+L} \sim \mathcal{N}(0, \sigma_c^2 \mathbf{I})$ is injected only on unmasked context tokens. That is, for positions with $m_i = 0$, the model conditions on slightly perturbed action tokens rather than perfectly clean ground-truth actions. This design mitigates the train-test mismatch between training, where unmasked tokens would otherwise be clean action tokens, and inference, where they are replaced by imperfect SFM predictions. It therefore reduces exposure bias and makes the model more robust to realistic context errors during AFM refinement.

In training, each element of $\mathbf{m}$ is identically and independently sampled from Bernoulli(y) with the pre-sampled probability $y \sim \mathcal{U}(0, 1)$. Following [Black et al., 2024], we sample the FM time τ from the Beta distribution $\text{Beta}(1.5, 1)$, which emphasizes noisier time steps close to 1. Note that when the sampled $\mathbf{m}$ is an all-1 vector, the AFM loss in Equation (4) degenerates to the vanilla SFM loss. Such training samples guarantee the unified model’s ability of both AFM and SFM inference. The randomly sampled $\mathbf{m}$ also equivalently plays the role of data augmentation that improves training data efficiency. The unified training procedure for AFM and SFM is summarized in Algorithm 2.

Training Confidence Rater After the VLA backbone is fully trained for SFM and AFM using Equation (4), we train the confidence rater with the frozen VLA backbone in an end-to-end manner. Since the actions generated by SFM do not provide direct signals that indicate the confidence of the model, we need to deliberately design the pseudo labels of the confidence rater. Define the sequence of token-wise mean-squared errors (MSEs) of the SFM-generated action chunk as $e_{t:t+L}$. Since the model should have higher confidence on tokens with smaller MSE and vice versa, we define the pseudo labels of the confidence rater as:

$$q_{t:t+L} = 1 - \alpha - \beta \frac{e_{t:t+L} - \min\{e_l\}}{\max\{e_l\} - \min\{e_l\} + \epsilon}, \quad (7)$$

where α and β are hyper-parameters that control the region of the pseudo labels in roughly $[1 - \alpha - \beta, 1 - \alpha]$, e.g., $[0.01, 0.99]$ if $\alpha = 0.01$ and $\beta = 0.98$, and ϵ is a small scalar that prevents the denominator being 0. In Equation (7), we alleviate the gradient vanishing problem caused by the final Sigmoid function, by preventing the labels from being extremely close to 0 or 1. To evaluate the confidence in a relative manner, $\max\{e_l\}$ and $\min\{e_l\}$ denote the maximum and minimum MSE in the action chunk, respectively. We thus assign the relatively accurate action tokens high confidence and utilize their context information to regenerate the action tokens with low confidence. When training the confidence rater, we set the loss function to the MSE between the confidence rater’s output and $q_{t:t+L}$. We empirically find that the labeling method in Equation (7) outperforms labeling by the task success indicator (TSI), i.e., $q_{t:t+L} = 1$ if the task is successful and $q_{t:t+L} = 0$ if the task is failed. We will analyze the reason through the ablation experiments in Section 4.3.

4 Simulation Experiments

4.1 Experimental Setup

We adopt Qwen2.5-VL-3B-Instruct [Bai et al., 2025] as the VLM backbone and augment it with an FM action head along with a confidence rater. Our AsyncVLA is pretrained on the Open X-

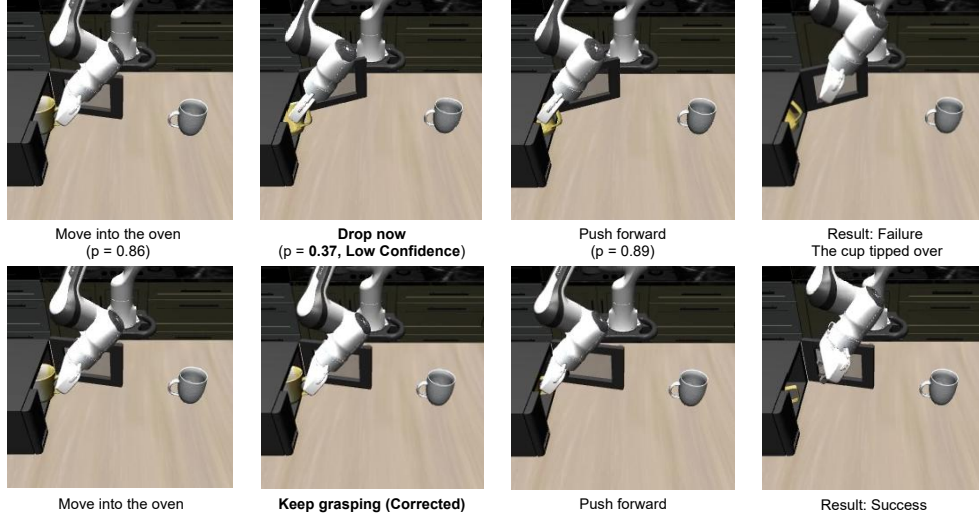


Figure 2: Illustration of self-correction ability in AsyncVLA on the LIBERO-Long task suite. The top row shows the first-round actions generated by SFM, and the bottom row shows the second-round actions regenerated by the following AFM.

Embodiment dataset [O’Neill et al., 2023] and is subsequently finetuned for different evaluation tasks on the corresponding datasets, including LIBERO [Liu et al., 2023], Bridge-V2 [Walke et al., 2023], and Fractal [Brohan et al., 2022]. In the data pre-processing stage, we mark the pause intervals in trajectories and exclude those action tokens from loss computation. The learning rate is set as 1×10^{-4} for both the language model backbone and the FM action head, and is set as 2×10^{-5} for the vision encoder. For unmasked context corruption in training, the noise scale is $\sigma_c = 0.05$. We use a 10-step denoising schedule for both SFM and AFM during inference. The backbone of the confidence rater is a 4-layer transformer with a linear layer as the rate head, which sums up to 308M parameters and takes up 7.56% of the total 4.08B parameters of the overall VLA model. In the mask generation, we set the confidence threshold as $T = 0.5$. When training the confidence rater, we set $\alpha = 0.01$, $\beta = 0.98$, and $\epsilon = 1 \times 10^{-6}$.

4.2 Evaluation Results

LIBERO Benchmark We finetune and evaluate AsyncVLA and baselines on the LIBERO benchmark [Liu et al., 2023]. Results are presented in Table 1, where we evaluate the models over 500 trials per task suite (10 tasks $\times$ 50 episodes). It is seen that AsyncVLA performs well in the LIBERO environment, achieving the highest success rates in all 4 tasks. By analyzing rollout videos of successful cases and comparing them with the trajectory generated by SFM in the same task, we find that AsyncVLA demonstrates the ability of self-correction, particularly in challenging tasks where first-round actions contain errors.

Self-Correction Ability We illustrate AsyncVLA’s self-correction ability on LIBERO-Long task in Figure 2. The top row shows the actions generated by SFM, and the bottom row shows the actions regenerated by the following AFM. Since the confidence rater gives a low confidence on the “drop now” action, this action token is remasked while others are not. AsyncVLA takes the other high-confidence action tokens into account as context and finds that before the “push forward” action, the correct action should be “keep grasping”. After AFM’s regeneration, the action with low confidence is corrected, and the task is successfully completed. It demonstrates that AFM with self-correction enhances AsyncVLA’s robustness to perturbations from first-round generated erroneous actions with large deviation.

WidowX Robot Benchmark We evaluate AsyncVLA and baselines on the WidowX Robot benchmark after further finetuning on the Bridge-V2 dataset [Walke et al., 2023]. We test across four generalization categories with environmental variations, and report the results in Table 2. AsyncVLA achieves the best overall performance, obtaining the highest average success rate (70.8%). Notably, it outperforms all baselines on three out of four tasks, including “put spoon on towel”, “put carrot on

Table 2: Comparison of different VLA models on the WidowX Robot benchmark conducted in SimplEnv [Liang et al., 2024].

Model	Put Spoon on Towel	Put Carrot on Plate	Stack Cubes	Put Eggplant in Basket	Avg.
OpenVLA [Kim et al., 2024]	0	0	0	4.1	1.0
SpatialVLA [Qu et al., 2025b]	20.8	20.8	25.0	70.8	34.4
Magma [Yang et al., 2025]	37.5	29.2	20.8	91.7	44.8
π_0 -FAST [Pertsch et al., 2025]	29.1	21.9	10.8	66.6	32.1
Octo-Base [Team et al., 2024]	12.5	8.3	0	43.1	16.0
Octo-Small [Team et al., 2024]	47.2	9.7	4.2	56.9	29.5
RoboVLM [Li et al., 2024b]	45.8	20.8	4.2	79.2	37.5
$\pi_{0.5}$ [Black et al., 2025]	49.3	64.7	44.7	69.7	57.1
ThinkAct [Huang et al., 2025a]	58.3	37.5	8.7	70.8	43.8
Discrete-Diffusion VLA [Liang et al., 2025]	37.5	29.2	20.8	—	—
UD-VLA [Chen et al., 2025a]	58.3	62.5	54.1	75.0	62.5
AsyncVLA (Ours)	70.8	66.7	58.3	87.5	70.8

Table 3: Comparison on the Google Robot benchmark under both visual matching (M) and variant aggregation (A) settings. The task “O/C Drawer” is short for “open or close the (top/middle/bottom) drawer”. Evaluation is conducted in SimplEnv [Liang et al., 2024].

Model	Pick Coke		Move Near		O/C Drawer		Put in Drawer		Avg.	
	M	A	M	A	M	A	M	A	M	A
TraceVLA [Zheng et al., 2025b]	28.0	60.0	53.7	56.4	57.0	31.0	0.0	0.0	34.7	36.9
SpatialVLA [Qu et al., 2025b]	86.0	88.0	77.9	72.7	57.4	41.8	0.0	6.3	55.3	52.2
Magma [Yang et al., 2025]	75.0	68.6	53.0	78.5	58.9	59.0	8.3	24.0	48.8	57.5
π_0 -FAST [Pertsch et al., 2025]	75.3	77.6	67.5	68.2	42.9	31.3	0.0	0.0	46.4	44.3
RT-2-X [Brohan et al., 2023]	78.7	82.3	77.9	79.2	25.0	35.3	7.4	20.6	47.3	54.4
Octo-Base [Team et al., 2024]	17.0	0.6	4.2	3.1	22.7	1.1	0.0	0.0	11.0	1.2
RoboVLM [Li et al., 2024b]	77.3	75.6	61.7	60.0	43.5	10.6	24.1	0.0	51.7	36.6
π_0 [Black et al., 2024]	97.9	90.1	78.7	80.7	62.3	27.6	46.6	20.5	71.4	54.7
ThinkAct [Huang et al., 2025a]	92.0	84.0	72.4	63.8	50.0	47.6	—	—	—	—
MolmoAct [Lee et al., 2025]	77.7	76.1	77.1	61.3	60.0	78.8	—	—	—	—
Discrete-Diffusion VLA [Liang et al., 2025]	85.4	82.5	67.5	64.6	60.6	23.6	—	—	—	—
AsyncVLA (Ours)	98.0	91.6	82.3	81.7	70.5	58.0	50.4	26.0	75.3	64.3

plate”, and “stack cubes”, demonstrating strong generalization across diverse manipulation skills. While Magma achieves the best performance on “put eggplant in basket”, AsyncVLA remains competitive on this task and maintains consistently strong performance across all categories.

Google Robot Benchmark We evaluate AsyncVLA and the baselines on the Google Robot benchmark after further finetuning on the Fractal dataset [Brohan et al., 2022]. As shown in Table 3, AsyncVLA consistently achieves the strongest overall performance, obtaining the highest average success rates under both visual matching (M) and variant aggregation (A) settings. Specifically, AsyncVLA achieves the best results on *Move Near* and *Put in Drawer* across both protocols, and also establishes new highs on *O/C Drawer* under the M setting.

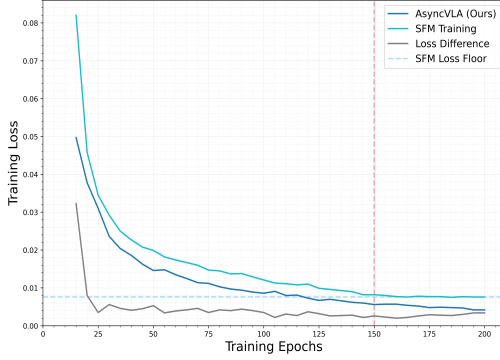
4.3 Ablation Study

We conduct ablation studies on four tasks from the WidowX Robot benchmark. We evaluate eight variants in total: “w/o Unified Training” removes the unified training scheme in Algorithm 2 and trains the model in the same way as vanilla SFM; “w/o AFM (10/20 steps)” removes AFM and uses SFM-only inference with 10 or 20 denoising steps; “w/o Confidence rater” keeps AFM but removes the confidence rater, where each action token is masked independently with probability 0.5; “w/ TSI labeling” trains the confidence rater using task-success-indicator (TSI) labels, i.e., $q_{t:t+L} = 1$ for successful trajectories and $q_{t:t+L} = 0$ otherwise; “w/ Delta refinement” replaces the AFM correction stage with a transformer-based refinement module that predicts action deltas; “w/ Direct refinement” replaces the AFM correction stage with a transformer-based refinement module that directly predicts refined actions; and “AsyncVLA (Ours)” denotes the complete method. Unless otherwise specified, both SFM and AFM use a 10-step denoising schedule.

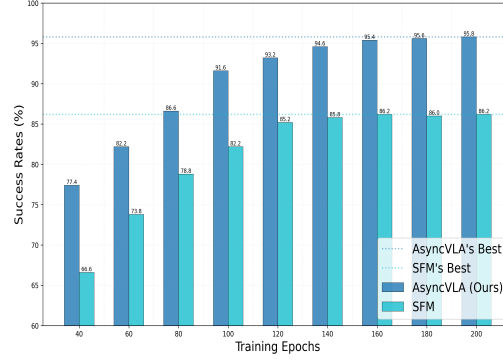
As shown in Table 4, AsyncVLA consistently achieves the best performance on all four tasks, reaching an average success rate of 70.8%. Removing unified training causes a dramatic drop to 7.3%, showing that aligning training with both synchronous and asynchronous inference is essential. Without AFM, performance drops to 47.9% with 10-step SFM and only recovers to 51.1% even

Table 4: Ablation study on the WidowX Robot benchmark in SimplifierEnv [Liang et al., 2024]. We use a 10-step denoising schedule in both SFM and AFM unless otherwise specified.

Model	Put Spoon on Towel	Put Carrot on Plate	Stack Cubes	Put Eggplant in Basket	Avg.
w/o Unified Training	4.2	8.3	0.0	16.7	7.3
w/o AFM (10 steps)	58.3	54.2	33.3	45.8	47.9
w/o AFM (20 steps)	58.3	54.2	37.5	54.2	51.1
w/o Confidence rater	66.7	54.2	54.2	75.0	62.5
w/ TSI labeling	66.7	58.3	54.2	79.2	64.6
w/ Delta refinement	66.7	58.3	45.8	75.0	61.5
w/ Direct refinement	66.7	54.2	50.0	79.2	62.5
AsyncVLA (Ours)	70.8	66.7	58.3	87.5	70.8



(a) Training loss curve comparison when only a quarter of the LIBERO-Spatial dataset is used for training.



(b) Success rate comparison in the training process. Evaluation is conducted on LIBERO-Spatial test suite.

with 20 SFM steps, indicating that simply increasing first-stage denoising is much less effective than adding a dedicated correction stage. Introducing AFM without a learned confidence rater already improves the average success rate to 62.5%, a gain of 14.6 points over SFM-only inference with 10 steps, which verifies the value of asynchronous self-correction itself. Refinement baselines further clarify where the gain comes from. Both Delta refinement (61.5%) and Direct refinement (62.5%) perform similarly to the random-mask variant and remain clearly below AsyncVLA. This suggests that the improvement is not merely due to adding a second refinement network. Instead, the key factor is the confidence-aware correction mechanism, which identifies unreliable action tokens and applies targeted asynchronous refinement to them. Finally, training the confidence rater with TSI labeling improves over random masking and the two generic refinement baselines, but still underperforms the full method (64.6% vs. 70.8%). The reason is that TSI provides only trajectory-level supervision and propagates the same binary outcome to all actions in the sequence, regardless of whether a particular step is helpful or harmful. Such coarse labels obscure token-level action quality and make it difficult for the confidence rater to learn fine-grained correction cues. In contrast, the dense labeling strategy in Equation (7) provides more precise supervision and leads to the strongest overall performance.

4.4 Training Data Efficiency

To demonstrate the data efficiency of the proposed unified training method in data-constrained settings, we separately train the models using AsyncVLA’s unified training method or vanilla SFM’s training method. The models are trained for 200 epochs with constant learning rates using a quarter of the LIBERO-Spatial dataset. The comparison of training loss curves is shown in Figure 3a, where all plotted loss is normalized per unmasked token. The training loss of AsyncVLA decreases faster and is remarkably and constantly lower than the training loss of SFM. Even when the training loss of SFM reaches a floor of 0.0076 and remains almost unchanged after 150 epochs, the AsyncVLA’s loss continues to decrease and ultimately reaches 0.0042 at the 200-th epoch.

We evaluate the success rates of AFM and SFM on LIBERO-Spatial test suite every 20 epochs in Figure 3b. AsyncVLA constantly outperforms SFM by at least 7.8%. When the success rate of SFM actually stops increasing and fluctuates around 86.2% after 140 epochs, the success rate of AsyncVLA still stably improves and finally reaches 95.8%. This demonstrates that AsyncVLA better



Figure 4: Real-world evaluated tasks on the AgileX PiPER robot. From left to right: (1) placing a carrot into a bowl; (2) taking a pen out of a pen holder; (3) pouring water from a cup into a bowl; and (4) placing a spoon onto a plate.

Table 5: Real-world success rates (%) on four manipulation tasks. Each task is evaluated over 50 trials on the AgileX PiPER robot.

Model	Carrot $\rightarrow$ Bowl	Pen Extraction	Pour Water	Spoon $\rightarrow$ Plate	Avg.
OpenVLA-OFT	82.0	68.0	46.0	66.0	65.5
π_0	72.0	76.0	48.0	64.0	65.0
$\pi_{0.5}$	86.0	78.0	68.0	76.0	77.0
AsyncVLA (Ours)	94.0	86.0	82.0	86.0	87.0

exploits the training data with longer training epochs in data-constrained settings. Such exploitation is attributed to the training data efficiency of the proposed unified training method that equivalently plays the role of data augmentation.

5 Real-World Experiments

To further validate our framework, we conduct extensive real-world experiments on the AgileX PiPER robot, as shown in Figure 4. The robot is equipped with a fixed front-view camera and a wrist camera for visual observation, and receives natural-language task instructions together with proprioceptive states as policy inputs. We evaluate four manipulation tasks covering precise grasping, object extraction, liquid pouring, and object placement. For each task, we conduct 50 trials under mild variations in object poses and initial states, and report the task-wise and average success rates.

As shown in Table 5, AsyncVLA achieves the best performance on all four tasks, reaching an average real-world success rate of 87.0%. The improvements are consistent across tasks and particularly pronounced on error-sensitive scenarios such as *pour water* and *pen extraction*, where slight action deviations can easily accumulate and cause failure. Compared with SFM baselines such as π_0 and $\pi_{0.5}$, AsyncVLA benefits from its confidence-guided asynchronous refinement mechanism, which selectively re-generates low-confidence action tokens while avoiding unnecessary changes to reliable ones, leading to more stable execution. Overall, these results demonstrate that AsyncVLA not only transfers effectively from simulation to real-world settings, but also provides a practical improvement in execution reliability for embodied manipulation.

6 Conclusion

In this work, we propose AsyncVLA, a novel framework that reframes action generation as a two-stage and confidence-aware process. Instead of using a fixed number of uniform denoising steps, AsyncVLA adaptively schedules the time steps in AFM. Moreover, we propose the confidence rater in AsyncVLA that estimates the relative confidence of each action token. Besides, we propose a unified training procedure for SFM and AFM, which endows a single model with both modes and improves KV-cache utilization. With the above improvement, AsyncVLA can dynamically reconsider its initially generated action tokens, focusing additional regeneration and asynchronous self-correction on the low-confidence components of each action chunk. Our extensive experiments demonstrate that AsyncVLA outperforms existing methods across both simulation and real-world evaluations.

References

- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL <https://arxiv.org/abs/2204.01691>.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibong Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- Kevin Black, Noah Brown, Danny Dries, Adnan Esmail, Michael Fiume, Chelsea Finn, Niccolo Fusi, Lachy Groom, Karol Hausman, Brian Ichter, and et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X. Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, Antoine Laurens, Claudio Fantacci, Valentin Dalibard, Martina Zambelli, Murilo Martins, Rugile Pevceviciute, Michiel Blokzijl, Misha Denil, Nathan Batchelor, Thomas Lampe, Emilio Parisotto, Konrad Zolna, Scott Reed, Sergio Gómez Colmenarejo, Jon Scholz, Abbas Abdolmaleki, Oliver Groth, Jean-Baptiste Regli, Oleg Sushkov, Tom Rothörl, José Enrique Chen, Yusuf Aytar, Dave Barker, Joy Ortiz, Martin Riedmiller, Jost Tobias Springenberg, Raia Hadsell, Francesco Nori, and Nicolas Heess. Robocat: A self-improving robotic agent. *arXiv preprint arXiv:2306.00287*, 2023.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Jaén, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, Jun 2025.
- Jiayi Chen, Wenxuan Song, Pengxiang Ding, Ziyang Zhou, Han Zhao, Feilong Tang, Donglin Wang, and Haoang Li. Unified diffusion VLA: Vision-language-action model via joint discrete denoising diffusion process. *arXiv preprint arXiv:2511.01718*, 2025a.
- Xinyi Chen, Yilun Chen, Yanwei Fu, Ning Gao, Jiaya Jia, Weiyang Jin, Hao Li, Yao Mu, Jiangmiao Pang, Yu Qiao, Yang Tian, Bin Wang, et al. InternVLA-M1: A spatially guided vision-language-action framework for generalist robot policy. *arXiv preprint arXiv:2510.13778*, 2025b.
- Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- StarVLA Community. StarVLA: A lego-like codebase for vision-language-action model developing. *GitHub repository*, 2025.
- Shaoqi Dong, Chaoyou Fu, Haihan Gao, Yi-Fan Zhang, Chi Yan, Chu Wu, Xiaoyu Liu, Yunhang Shen, Jing Huo, Deqiang Jiang, Haoyu Cao, Yang Gao, Xing Sun, Ran He, and Caifeng Shan. VITA-VLA: Efficiently teaching vision-language models to act via action expert distillation. *arXiv preprint arXiv:2510.09607*, 2025.
- Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, and Jianlan Luo. Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation. *arXiv preprint arXiv:2502.16707*, 2025.

- Zipeng Fu, Tony Z. Zhao, and Chelsea Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. *arXiv preprint arXiv:2401.02117*, 2024.
- Dechen Gao, Boqi Zhao, Andrew Lee, Ian Chuang, Hanchu Zhou, Hang Wang, Zhe Zhao, Junshan Zhang, and Iman Soltani. Vita: Vision-to-action flow matching policy. *arXiv preprint arXiv:2507.13231*, 2025.
- Ankit Goyal, Hugo Hadfield, Xuning Yang, Valts Blukis, and Fabio Ramos. VLA-0: Building state-of-the-art VLAs with zero modification. *arXiv preprint arXiv:2510.13054*, 2025.
- Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. ThinkAct: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025a.
- Jiannan Huang, Ding Ding, Zhixing Tang, Kai Liu, Yunhai Chen, Pengcheng He, and Bin Yang. A survey on integration of large language models with intelligent robots. *arXiv preprint arXiv:2404.09228*, 2024.
- Wenhui Huang, Changhe Chen, Han Qi, Chen Lv, Yilun Du, and Heng Yang. MoTVLA: A vision-language-action model with unified fast-slow reasoning. *arXiv preprint arXiv:2510.18337*, 2025b.
- Zhiling Huang, Yuke Zhu, Fei Xia, and Manolis Savva. Open-ended language-guided planning for vision-and-language navigation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 18779–18790, 2023.
- Yuhua Jiang, Shuang Cheng, Yihao Liu, Ermo Hua, Che Jiang, Weigao Sun, Yu Cheng, Feifei Gao, Biqing Qi, and Bowen Zhou. Nirvana: A specialized generalist model with task-aware memory mechanism. *arXiv preprint arXiv:2510.26083*, 2025.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Rafael Rafailov, Ananya P. Foster, Pannag R. Sanketi, Quan Vuong, Sergey Levine, and et al. OpenVLA: An open-source vision-language-action model. In *8th Annual Conference on Robot Learning*, 2024.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, Winson Han, Wilbert Pumacay, Angelica Wu, Rose Hendrix, Karen Farley, Eli VanderBilt, Ali Farhadi, Dieter Fox, and Ranjay Krishna. MolmoAct: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.
- Baicheng Li, Dong Wu, Zike Yan, Xincheng Liu, Zecui Zeng, Lusong Li, and Hongbin Zha. Reflection-based task adaptation for self-improving VLA. *arXiv preprint arXiv:2510.12710*, 2025a.
- Chenxuan Li, Jiaming Liu, Guanqun Wang, Xiaoqi Li, Sixiang Chen, Liang Heng, Chuyan Xiong, Jiabin Ge, Renrui Zhang, Kaichen Zhou, and Shanghang Zhang. A self-correcting vision-language-action model for fast and slow system manipulation. *arXiv preprint arXiv:2405.17418*, 2025b.
- Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025c.
- Peiyan Li, Yixiang Chen, Hongtao Wu, Xiao Ma, Xiangnan Wu, Yan Huang, Liang Wang, Tao Kong, and Tieniu Tan. BridgeVLA: Input-output alignment for efficient 3d manipulation learning with vision-language models. *arXiv preprint arXiv:2506.07961*, 2025d.
- Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Siheng Xu, Yizhong Zhang, and et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024a.
- Xinghang Li, Peiyan Li, Minghuan Liu, Dong Wang, Jirong Liu, Bingyi Kang, Xiao Ma, Tao Kong, Hanbo Zhang, and Huaping Liu. Towards generalist robot policies: What matters in building vision-language-action models. *arXiv preprint arXiv:2412.14058*, 2024b.
- Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiabin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhiwei Li, Bao-Long Bi, Ling-Rui Mei, Junfeng Fang, Xiao Liang, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*, 2025e.
- Xuanlin Liang, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walk, Chuyuan Lunawat, Isabel Ishikaa, Sean Kimani, Sergey Levine, and et al. Evaluating real-world robot manipulation policies in simulation. In *Conference on Robot Learning*, pages 3705–3728, 2024.

- Zhixuan Liang, Yizhuo Li, Tianshuo Yang, Chengyue Wu, Sitong Mao, Liyao Pei, Xiaokang Yang, Jiangmiao Pang, Yao Mu, and Ping Luo. Discrete diffusion VLA: Bringing discrete diffusion to action decoding in vision-language-action policies. *arXiv preprint arXiv:2508.20072*, 2025.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. URL <https://arxiv.org/abs/2210.02747>.
- Bo Liu, Yifeng Yuan, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Han, and Peter Stone. Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 44776–44791, 2023.
- Haoning Liu, Shuqiang Liu, Jun Song, Guozheng Zhang, Hong Liu, and Jianwen Zhang. A review of foundation models for vision, language and action in robotics. *arXiv preprint arXiv:2402.17643*, 2024.
- Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can RL bring to VLA generalization? An empirical study. *arXiv preprint arXiv:2505.19789*, 2025.
- Qi Lv, Weijie Kong, Hao Li, Jia Zeng, Zherui Qiu, Delin Qu, Haoming Song, Qizhi Chen, Xiang Deng, and Jiangmiao Pang. F1: A vision-language-action model bridging understanding and generation to actions. *arXiv preprint arXiv:2509.06951*, 2025. URL <https://arxiv.org/abs/2509.06951>.
- Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2401.07172*, 2024.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. URL <https://arxiv.org/abs/2502.09992>.
- NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi ”Jim” Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.17434*, 2024.
- Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Acorn Pooley, Arijit Gupta, Ajay Mandelkar, Ajinkya Jain, et al. Open X-Embodiment: Robotic learning datasets and RT-X models. *arXiv preprint arXiv:2310.08864*, 2023.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Dries, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Delin Qu, Haoming Song, Qizhi Chen, Zhaoqing Chen, Xianqiang Gao, Modi Shi, Guanghui Ren, Maoqing Yao, Bin Zhao, and Dong Wang. EmbodiedOneVision: Interleaved vision-text-action pretraining for general robot control. *arXiv preprint arXiv:2508.21112*, 2025a.
- Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Jiayuan Wang, Bin Gu, and Zhiqiang Zhao. SpatialVLA: Exploring spatial representations for visual language-action model. *arXiv preprint arXiv:2501.15830*, 2025b.
- Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yuri Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. A generalist agent. *arXiv preprint arXiv:2205.06175*, 2022.
- Ranjan Sapkota, Yang Cao, and Manoj Karkee. Vision-language-action models: Concepts, progress, applications and challenges. *arXiv preprint arXiv:2505.04769*, 2025.
- Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.
- Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

- Nan Sun, Yongchang Li, Chenxu Wang, Huiying Li, and Huaping Liu. CollabVLA: Self-reflective vision-language-action model dreaming together with human. *arXiv preprint arXiv:2509.14889*, 2025.
- Alexander Swerdlow, Mihir Prabhudesai, Siddharth Gandhi, Deepak Pathak, and Katerina Fragkiadaki. Unified multimodal discrete diffusion. *arXiv preprint arXiv:2503.20853*, Mar 2025.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. *Robotics: Science and Systems*, 2024.
- Homer Rich Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Maximilian Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Ho Vuong, Andre Wang He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. *arXiv preprint arXiv:2310.03816*, Oct 2023.
- Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, Siteng Huang, Yifan Tang, Wenhui Wang, Ru Zhang, Jianyi Liu, and Donglin Wang. VLA-Adapter: An effective paradigm for tiny-scale vision-language-action model. *arXiv preprint arXiv:2509.09372*, 2025.
- Junjie Wen, Minjie Zhu, Jiaming Liu, Zhiyuan Liu, Yicun Yang, Linfeng Zhang, Shanghang Zhang, Yichen Zhu, and Yi Xu. dVLA: Diffusion vision-language-action model with multimodal chain-of-thought. *arXiv preprint arXiv:2509.25681*, 2025a.
- Junjie Wen, Minjie Zhu, Yichen Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Chengmeng Li, Xiaoyu Liu, Yaxin Peng, Chaomin Shen, and Feifei Feng. Diffusion-VLA: Generalizable and interpretable robot foundation model via self-generated reasoning. *arXiv preprint arXiv:2412.03293*, 2025b.
- Yuqing Wen, Hebei Li, Kefan Gu, Yucheng Zhao, Tiancai Wang, and Xiaoyan Sun. LLaDA-VLA: Vision language diffusion action models. *arXiv preprint arXiv:2509.06932*, 2025c.
- Zhenyu Wu, Yuheng Zhou, Xiuwei Xu, Ziwei Wang, and Haibin Yan. MoManipVLA: Transferring vision-language-action models for general mobile manipulation. *arXiv preprint arXiv:2503.13446*, 2025.
- Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, Yuquan Deng, Lars Liden, and Jianfeng Gao. Magma: A foundation model for multimodal ai agents. *arXiv preprint arXiv:2502.13130*, 2025.
- Chao Yu, Yuanqing Wang, Zhen Guo, Hao Lin, Si Xu, Hongzhi Zang, Quanlu Zhang, Yongji Wu, Chunyang Zhu, Junhao Hu, Zixiao Huang, Mingjie Wei, Yuqing Xie, Ke Yang, Bo Dai, Zhexuan Xu, et al. RLinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025. URL <https://arxiv.org/abs/2509.15965>.
- Hongzhi Zang, Mingjie Wei, Si Xu, Yongji Wu, Zhen Guo, Yuanqing Wang, Hao Lin, Liangzhi Shi, Yuqing Xie, Zhexuan Xu, Zhihao Liu, et al. RLinf-VLA: A unified and efficient framework for VLA+RL training. *arXiv preprint arXiv:2510.06710*, 2025.
- Andy Zhai, Brae Liu, Bruno Fang, Chalse Cai, Ellie Ma, Ethan Yin, Hao Wang, Hugo Zhou, James Wang, Lights Shi, Lucy Liang, Make Wang, Qian Wang, Roy Gan, Ryan Yu, Shalfun Li, Starrick Liu, Syllas Chen, Vincent Chen, and Zach Xu. Igniting vlms toward the embodied space. *arXiv preprint arXiv:2509.11766*, 2025.
- Qinglun Zhang, Zhen Liu, Haoqiang Fan, Guanghui Liu, Bing Zeng, and Shuaicheng Liu. Flowpolicy: Enabling fast and robust 3d flow-based policy via consistency flow matching for robot manipulation. *arXiv preprint arXiv:2412.04987*, 2024.
- Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *CVPR*, 2024.
- Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyu Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025a. URL <https://arxiv.org/abs/2510.10274>.
- Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025b.

Zhide Zhong, Haodong Yan, Junfeng Li, Xiangchen Liu, Xin Gong, Tianran Zhang, Wenxuan Song, Jiayi Chen, Xihu Zheng, Hesheng Wang, and Haoang Li. FlowVLA: Visual chain of thought-based motion reasoning for vision-language-action models. *arXiv preprint arXiv:2508.18269*, 2025.

A Technical Appendices and Supplementary Material

A.1 Limitations and Future Works

One limitation of the current confidence-labeling design is that the pseudo-labels for training the confidence rater are defined by min-max normalization within each action chunk. This relative formulation is effective for identifying tokens that are less reliable than others in the same chunk, but it may suffer from a corner case when the initial SFM prediction fails globally. In such cases, even the token with the smallest error in the chunk may still have a large absolute error, yet it can be assigned a high confidence score and thus be preserved as context during asynchronous regeneration. In future work, we plan to address this issue by incorporating absolute error-aware calibration, such as combining relative token-wise confidence with a chunk-level reliability estimate or an absolute error threshold, so that globally unreliable predictions can be more conservatively refined.

Although AsyncVLA achieves strong performance on VLA tasks, its current empirical validation is still limited to robotic action generation. We have not yet examined whether the proposed asynchronous generation and confidence-guided refinement mechanism can generalize to broader generative domains, such as image generation or other multimodal generation tasks. Exploring the applicability of AsyncVLA beyond VLA models is an important direction for future work, and we expect that the same principle of selectively refining low-confidence components may provide a useful mechanism for more general generation scenarios.

A.2 Statement for Use of LLMs

LLMs were only used to assist with language polishing in certain sections of this paper.

A.3 Impact Statement

This paper presents work whose goal is to advance the field of machine learning. We do not identify any specific impacts of this work that require particular emphasis here.

A.4 Reproducibility Statement

We provide open-source code to reproduce all experiments, including training and evaluation. The repository contains the full implementation of the SFM, confidence rater, and AFM pipelines. Code is available at <https://github.com/YuhuaJiang2002/AsyncVLA>.

B Inference Time Analysis

Table 6: Wall-clock inference time of each component on an NVIDIA GeForce RTX 4090 GPU with 2 camera images.

Component	Inference Time	Percentage
SFM	83.2 ± 1.4 ms	86.8%
Confidence rater	2.6 ± 0.1 ms	2.7%
AFM	10.1 ± 0.3 ms	10.5%
Overall	95.9 ± 1.6 ms	100.0%

The wall-clock inference time of each component is shown in Table 6, which is timed on an NVIDIA GeForce RTX 4090 GPU with 2 camera images and 10 FM steps. During inference, the majority of the computational cost is incurred by SFM. In SFM, the model computes a full forward pass over all VL and action tokens. This includes constructing the entire VL KV-cache from scratch. Consequently, SFM accounts for 86.8% of the total inference time. In contrast, AFM is substantially more efficient because it reuses the VL KV-cache produced in the very first pass. After SFM initializes the KV-cache, AFM does not recompute these caches in subsequent steps. Instead, it only updates the subset of action tokens whose confidence is below the threshold. The confidence rater introduces 2.7% overhead, since it is executed only once per action chunk, produces a lightweight scalar confidence

value for each action token, and does not participate in iterative FM-style updates. Overall, the reuse of VL KV-cache and the partial-token update mechanism explain why AFM inference is significantly faster than SFM, while still enabling self-correction and higher success rates.

C Training and Inference Pseudo codes

Algorithm 1 Asynchronous Flow Matching Inference

Require: $\mathbf{o}_t, \ell, \hat{\mathbf{a}}_{t:t+L}^{\text{SFM}}, \mathbf{m} \in \{0, 1\}^L$; step size δ

- 1: For all l with $m_l = 0$: set $\hat{\mathbf{a}}_l^1 \leftarrow \hat{\mathbf{a}}_l^{\text{SFM}}$;
- 2: For all l with $m_l = 1$: set $\hat{\mathbf{a}}_l^1 \leftarrow \mathbf{n}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 3: Set $\tau \leftarrow 1$
- 4: **while** $\tau > 0$ **do**
- 5: Predict velocity: $\hat{\mathbf{v}}_{t:t+L} \leftarrow V_\theta(\mathbf{o}_t, \ell, \hat{\mathbf{a}}_{t:t+L}^\tau)$
- 6: For all l with $m_l = 0$: $\hat{\mathbf{a}}_l^{\tau-\delta} \leftarrow \hat{\mathbf{a}}_l^\tau$
- 7: For all l with $m_l = 1$: $\hat{\mathbf{a}}_l^{\tau-\delta} \leftarrow \hat{\mathbf{a}}_l^\tau - \delta \hat{\mathbf{v}}_l$
- 8: $\tau \leftarrow \max(0, \tau - \delta)$
- 9: **end while**
- 10: **return** Final action $\tilde{\mathbf{a}}_{t:t+L} = \hat{\mathbf{a}}_{t:t+L}^0$

Algorithm 2 Unified Training for AFM and SFM

Require: Dataset $\mathcal{D}$; FM model V_θ ; batch size B

- 1: **repeat**
- 2: Sample $\{(\mathbf{o}_t^{(i)}, \mathbf{a}_{t:t+L}^{(i)}, \ell^{(i)})\}_{i=1}^B \sim \mathcal{D}$
- 3: **for** $i = 1, \dots, B$ **do**
- 4: Sample $y^{(i)} \sim \mathcal{U}(0, 1); m_l^{(i)} \sim \text{Bernoulli}(y^{(i)})$
- 5: Sample $\tau^{(i)} \sim \text{Beta}(1.5, 1); \mathbf{n}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 6: Sample $\boldsymbol{\epsilon}^{(i)} \sim \mathcal{N}(\mathbf{0}, \sigma_c^2 \mathbf{I})$
- 7: $\mathbf{u}^{(i)} \leftarrow \mathbf{n}^{(i)} - \mathbf{a}^{(i)}$
- 8: $\tilde{\mathbf{a}}^{(i)} \leftarrow \mathbf{a}_{t:t+L}^{(i)} + \boldsymbol{\epsilon}^{(i)} \odot (\mathbf{1} - \mathbf{m}^{(i)})$
- 9: $\hat{\mathbf{a}}^{\tau^{(i)}} \leftarrow \tilde{\mathbf{a}}^{(i)} - \tau^{(i)} (\tilde{\mathbf{a}}^{(i)} - \mathbf{n}_{t:t+L}^{(i)}) \odot \mathbf{m}^{(i)}$
- 10: $\hat{\mathbf{v}}^{(i)} \leftarrow V_\theta(\mathbf{o}_t^{(i)}, \ell^{(i)}, \hat{\mathbf{a}}^{\tau^{(i)}})$
- 11: **end for**
- 12: $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^B \|\hat{\mathbf{v}}^{(i)} - \mathbf{u}^{(i)}\| \odot \mathbf{m}^{(i)}\|^2$
- 13: Update θ by back-propagation on $\mathcal{L}$
- 14: **until** converged

D AsyncVLA vs. Discrete Diffusion VLA

Table 7: Comparison between Discrete Diffusion VLA and AsyncVLA.

Aspect	Discrete Diffusion VLA	AsyncVLA
Action type	Discrete	Continuous
Prediction parameterization	Target (x -prediction)	Velocity (v -prediction)
ReMasking mechanism	Mask token	Gaussian noise
Confidence computation	Logits	Confidence rater

Table 7 summarizes the key differences between AsyncVLA and discrete diffusion VLA (such as LLaDA-VLA [Wen et al., 2025c], dVLA [Wen et al., 2025a], discrete-diffusion VLA [Liang et al., 2025], and UD-VLA [Chen et al., 2025a]) in four dimensions: action representation, prediction parameterization, remasking mechanism, and confidence estimation. Discrete diffusion VLA operates

on *discrete* action tokens and typically adopts *target prediction* (i.e., x -prediction) under a *mask-token* corruption scheme, where confidence can be directly derived from the model’s *logits*. In contrast, AsyncVLA models *continuous* actions and performs *velocity prediction* (v -prediction) under *Gaussian noise* masking; its confidence is computed by a separate *confidence rater* rather than raw logits. As demonstrated in our experiments, AsyncVLA achieves a significantly higher success rate compared to discrete diffusion VLA, further validating the superiority of continuous action representations and Gaussian noise masking in improving model performance and generalization ability.

Although this paper primarily uses v -prediction for its favorable optimization and sampling properties, the AsyncVLA formulation can be extended to alternative diffusion parameterizations with minimal changes. Concretely, the asynchronous masking-and-reconstruction pipeline, the noise-corruption process, and the confidence rater interface remain unchanged; one only needs to swap the denoiser head (and the corresponding training target) from predicting v to predicting either the clean target x (x -prediction) or the added noise ϵ (ϵ -prediction). Since v , x , and ϵ are deterministically related given the diffusion schedule, the same model architecture and sampling loop can be reused by replacing the prediction type and applying the standard conversion between parameterizations during training and inference.

Table 8: Dataset Weights

Dataset	Weight
Bridge-V2	24.14%
RT-1	13.80%
TOTO	10.34%
VIOLA	10.34%
RoboTurk	10.34%
Jaco Play	10.34%
Berkeley Autolab UR5	10.34%
Berkeley Fanuc Manipulation	10.34%

D.1 Fine-Grained Parameter Exploration on LIBERO

To examine the robustness of AsyncVLA to the hyperparameters of the confidence-aware refinement mechanism, we perform a fine-grained parameter study on LIBERO. Starting from the default setting ($\alpha = 0.01, \beta = 0.98, T = 0.5$), we vary one parameter at a time while keeping the others fixed. Specifically, we test a larger positive-target coefficient $\alpha = 0.05$, a smaller negative-target coefficient $\beta = 0.95$, and two alternative refinement thresholds $T = 0.25$ and $T = 0.75$.

As shown in Table 9, varying one parameter at a time leads to only minor changes in performance. The average success rate remains within a narrow range of 97.2–97.4, while all four LIBERO suites stay highly stable. These results indicate that AsyncVLA is not sensitive to these hyperparameters. In particular, changing α and β only mildly affects the soft supervision targets used for training the confidence rater, whereas changing T only adjusts the threshold for selecting low-confidence tokens for refinement. Overall, the method exhibits robust behavior across a reasonable range of parameter choices.

E Implementation Details

E.1 Details of Training

In Table 8, we summarize the detailed datasets and their weights during the pre-training stage of AsyncVLA. We select part of the Open X-Embodiment dataset [O’Neill et al., 2023] as our pre-training robot demonstration data. We perform pre-training on 4 H200 GPU nodes (8 GPUs per node, 32 GPUs in total) under BF16 precision with gradient checkpointing enabled. We use flash-attention-2 and ZeRO-2 optimizer sharding for efficient training and proper GPU memory usage. The global batch size is set to 2048. The pre-training process takes roughly 2.5 days. We use the cosine decay learning rate scheduler with the largest learning rate set as 1×10^{-4} . Further

Table 9: Fine-grained parameter exploration of AsyncVLA on LIBERO. We vary one parameter at a time from the default setting ($\alpha = 0.01, \beta = 0.98, T = 0.5$). Results are reported in success rate (%).

Model	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-Long	Avg.
AsyncVLA (Ours)	98.4	99.2	98.6	93.4	97.4
$\alpha = 0.05$	98.2	99.0	98.4	93.0	97.2
$\beta = 0.95$	98.4	99.2	98.2	93.2	97.3
$T = 0.25$	98.2	99.2	98.2	93.0	97.2
$T = 0.75$	98.6	99.0	98.4	93.0	97.3

fine-tuning on LIBERO, Bridge-V2, and Fractal is performed on a single H200 node with 8 GPUs, requiring 15–32 hours depending on the dataset size. The chat template includes the prompt “You are a helpful physical assistant.” at the beginning of each sample. Throughout all stages, we use AdamW optimizer with weight decay set to 0, $\beta_1 = 0.9$, and $\beta_2 = 0.999$. In the mask generation, we set the confidence threshold as $T = 0.5$. When training the confidence rater, we set $\alpha = 0.01$, $\beta = 0.98$, and $\epsilon = 1 \times 10^{-6}$.

E.2 Details of Flow Matching

For both SFM and AFM, we use a uniform schedule of 10 discretization steps from noise at $\tau = 1$ towards the target action at $\tau = 0$. This choice provides a balanced trade-off between computational efficiency and control performance, yielding stable convergence and high success rates while keeping inference latency low. In practice, we find that increasing the number of steps offers diminishing returns, whereas fewer steps noticeably degrade action accuracy and long-horizon stability.

E.3 Structure of Confidence Rater

The backbone of the confidence rater is a 4-layer transformer with a linear layer rate head, which sums up to 308M parameters and takes up 7.56% of the total 4.08B parameters of the overall VLA model. Each of the four transformer blocks contains multi-head self-attention with 32 attention heads and a feed-forward network width of 6144.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope.

Guidelines:

- The answer [\[NA\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[NA\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Limitations are discussed in the Appendix.

Guidelines:

- The answer [\[NA\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[NA\]](#)

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [NA] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: All the information needed to reproduce the main experimental results of the paper is shown in the 3rd and 4th sections.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to the data and code in the Reproducibility Statement Section

Guidelines:

- The answer [NA] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: All the training and test details are shown in the Experiments section.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We report the number of trials in real-world experiments.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper provides sufficient information on the computer resources in Appendix.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics.

Guidelines:

- The answer [NA] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: the paper discuss both potential positive societal impacts and negative societal impacts of the work performed.

Guidelines:

- The answer [NA] means that there is no societal impact of the work performed.
- If the authors answer [NA] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: the paper poses no such risks.

Guidelines:

- The answer [NA] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected.

Guidelines:

- The answer [NA] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: Codes introduced in the paper are well documented in the anonymous code link.

Guidelines:

- The answer [\[NA\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[NA\]](#)

Justification: the paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [\[NA\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[NA\]](#)

Justification: the paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [\[NA\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.

- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [\[Yes\]](#)

Justification: the paper describe the usage of LLMs in the appendix.

Guidelines:

- The answer [\[NA\]](#) means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.